

Day 2 — Memoization Bridge + Light Backtracking

Moving fast from plain recursion into DP thinking, with backtracking kept as one light choice-tree example.

Medium difficulty · **80-minute class** · **Recursion → Memoization**

The Notebook Analogy



Ask yourself: what is 7×8 ?

You work it out once — 56. If someone asks ten more times, do you recompute it each time?

Plain recursion is the student who recomputes 7×8 every single time.

Memoization is keeping a notebook: solve each question once, jot the answer, and just read it back next time.

Today you'll teach your code to keep that notebook.

THE ANALOGY, MAPPED TO CODE

The question asked → the state (what changes)

Your notebook → the cache / dp array

The jotted answer → the stored result `dp[state]`

"I've seen this one" → the cache-hit check

Day 2 at a Glance

TODAY'S TOPIC

Memoization Bridge + Light Backtracking

WHY THIS DAY EXISTS

Day 1 ended by showing that repeated recursive calls are exactly why DP is needed.

Day 2 moves students quickly from plain recursion into DP thinking — using backtracking only as one “choice tree” example, not a deep topic.

CORE SKILL — STUDENTS MUST UNDERSTAND

- Repeated (overlapping) subproblems
- The cache key that identifies a subproblem
- Top-down DP = recursion + a cache
- State = “what changes” between calls

Class Roadmap

1



Pattern Trigger

Show why plain recursion becomes slow

2



Core Concept

State, cache key, and the memo table

3



Guided Problem 1

Fibonacci: recursion → memoization

4



Guided Problem 2

Min Cost Climbing Stairs memoization

5



Light Backtracking

One subset / choice-tree example

6



Exit Ticket

Identify what changes and what to cache

Pattern Trigger — Spotting Repeated Work

DP doesn't start with tables. It starts when we notice the same question asked again and again.

TWO QUESTIONS FOR EVERY RECURSION

- What changes?
- Can we store the answer for that changing value?

ASK THE CLASS

- In Fibonacci, which calls repeated yesterday?
- If fib(3) is already solved, why solve it again?
- If only n changes, what is the cache key?

BOARD — CORE QUESTIONS

1. What changes?
2. What is the cache key?
3. What answer should be stored?

Write these three on the board and leave them up all class — every problem today answers them.

Repeated Subproblems & State

FIB(5) RECURSION TREE

```
fib(5)
├── fib(4)
│   ├── fib(3)
│   │   ├── fib(2)
│   │   └── fib(1)
│   └── fib(2)
└── fib(3)
```

Repeated: fib(3), fib(2), fib(1)

Repeated work + same answer = a caching opportunity.

STATE = WHAT CHANGES

fib(n) → what changes? **n** → cache dp[n]

cost(i) → what changes? **i** → cache dp[i]

KEY MESSAGE

This is precisely the definition of overlapping subproblems: the same state appears more than once, and its answer never changes.

Cache Key & the Top-Down DP Template

CACHE KEY MAPS STATE → STORAGE

State	Cache key	Stores
n	dp[n]	fib(n)
i	dp[i]	minCost(i)
i, j	dp[i][j]	answer(i, j)

Warning: cache the function's meaning — never a random value.

TOP-DOWN DP TEMPLATE

```
solve(state):  
  1. base case    → return answer  
  2. if dp[state] → return it  
  3. compute from smaller states  
  4. store dp[state]  
  5. return dp[state]
```

```
int[] dp = new int[n + 1];  
Arrays.fill(dp, -1);
```

Use -1 as the “empty” marker only when valid answers are non-negative.

Fibonacci Memoization

PROBLEM

Given n , return the n th Fibonacci number.

```
fib(0) = 0    fib(1) = 1
fib(n) = fib(n-1) + fib(n-2)
```

DEFINE BEFORE CODE

Meaning: $\text{fib}(n)$ = n th Fibonacci

State: n

Cache key: n

Store: $\text{dp}[n] = \text{fib}(n)$

MAIN.JAVA

```
import java.util.*;
class Main {
    static int fib(int n, int[] dp) {
        if (n == 0) return 0;
        if (n == 1) return 1;
        if (dp[n] != -1) return dp[n];
        dp[n] = fib(n-1, dp) + fib(n-2, dp);
        return dp[n];
    }
    public static void main(String[] a) {
        int n = 6;
        int[] dp = new int[n + 1];
        Arrays.fill(dp, -1);
        System.out.println(fib(n, dp));
    }
}
```


Fibonacci — Dry Run & Complexity

DRY RUN — FIB(5)

```
fib(5) needs fib(4), fib(3)
fib(4) needs fib(3), fib(2)
fib(3) needs fib(2), fib(1)
fib(2) needs fib(1), fib(0)
  store dp[2] = 1
store dp[3] = 2
store dp[4] = 3
fib(3) needed again →
  dp[3] exists → return 2
store dp[5] = 5
```

Same recurrence — memoization only avoids recomputation.

COMPLEXITY

PLAIN RECURSION

Exponential

time · $O(n)$ recursion stack

MEMOIZED RECURSION

$O(n)$ time

$O(n)$ dp + $O(n)$ recursion stack

Min Cost Climbing Stairs — Modelling

PROBLEM

$\text{cost}[i]$ is the cost of stepping on stair i . You may climb 1 or 2 steps at a time. Find the minimum cost to reach the top.

$\text{minCost}(i)$ = minimum cost to reach step i . The last move reaches i from $i-1$ or $i-2$.

```
minCost(i) = cost[i]
            + min(minCost(i-1),
                  minCost(i-2))
```

MODEL A — cost to stand on each stair

Answer = $\min(\text{minCost}(n-1), \text{minCost}(n-2))$

The top is after the last stair.

MODEL B — cost to reach a virtual top (use this)

```
solve(i) = min(
    solve(i-1) + cost[i-1],
    solve(i-2) + cost[i-2])

Answer = solve(n)
```

Min Cost — Code & Dry Run

MAIN.JAVA

```
static int solve(int i, int[] cost,
                 int[] dp) {
    if (i == 0 || i == 1) return 0;
    if (dp[i] != -1) return dp[i];
    int one = solve(i-1, cost, dp)
               + cost[i-1];
    int two = solve(i-2, cost, dp)
               + cost[i-2];
    dp[i] = Math.min(one, two);
    return dp[i];
}
```

DRY RUN — COST = [10, 15, 20]

```
solve(2)
= min(0+15, 0+10) = 10

solve(3)
= min(10+20, 0+15)
= min(30, 15) = 15
```

15

minimum cost to reach the top

Light Backtracking — Subset Choice Tree

ONE IDEA ONLY

Don't teach full backtracking today — show **one idea** with the array [2, 3, 5]: print all subsets. At each index there are two choices — **pick** or **skip** the current element.

Backtracking explores choices.

Memoization remembers repeated states.

DP = recursion + repeated states + stored answers.

CHOICE TREE — [2, 3, 5]

```
index 0: 2
├── pick 2
│   ├── pick 3
│   │   ├── pick 5 → [2,3,5]
│   │   └── skip 5 → [2,3]
│   └── skip 3
│       ├── pick 5 → [2,5]
│       └── skip 5 → [2]
└── skip 2
    ├── pick 3
    │   ├── pick 5 → [3,5]
    │   └── skip 5 → [3]
    └── skip 3
        ├── pick 5 → [5]
        └── skip 5 → [ ]
```

Exit Ticket

PROMPT

Given this recursive idea:

```
ways(n) = ways(n-1) + ways(n-2)
```

- What is the function meaning?
- What changes?
- What is the cache key?
- What should `dp[n]` store?
- Where should the memo check go?

REVEAL AFTER STUDENTS ANSWER

Meaning: number of ways to reach step `n`

Changes: `n`

Cache key: `n`

`dp[n]` stores: ways to reach step `n`

Memo check: after base cases, before the recursive calls

Homework

A1 · MUST DO**Fibonacci Memoization**

Write memoized Java. Also list meaning, state, base case, transition, cache key, and repeated calls avoided.

A2 · MUST DO**Min Cost Climbing Stairs**

Write top-down DP. Explain why $\text{solve}(0)=0$ and $\text{solve}(1)=0$, and what $\text{dp}[i]$ stores.

B1 · RECURRENCE ONLY**Minimum Moves to Reach 0**

Moves: $n \rightarrow n-1$, $n \rightarrow n-3$, $n \rightarrow n-5$. Give meaning, state, base, transition, cache key.

$$\text{moves}(n) = 1 + \min(\text{moves}(n-1), \text{moves}(n-3), \text{moves}(n-5))$$

B2 · RECURRENCE ONLY**Count Ways to Reach Score**

A move scores 1, 2, or 4 points. Count ways to reach exactly score n . Give the memo key.

$$\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2) + \text{ways}(n-4)$$

Common Mistakes to Track



“DP means tables only”

→ Top-down DP is recursion plus a cache.



Can’t define the state

→ Ask: what changes between recursive calls?



Caching the wrong value

→ Cache exactly what the function returns.



Forgetting base cases

→ Base cases must match the function’s meaning.



Memoizing before the recurrence

→ Recurrence first, memoization second.



Assuming all recursion needs memo

→ Only helps when the same state repeats.

Day 2 — The Big Picture

BOARD SUMMARY

Plain recursion solves repeated subproblems again & again

Memoization stores the answer after the first solve

State what changes?

Cache key how we identify the subproblem

Template base · cache check · recurse · store · return

Key idea same state, same answer? store it.

SUCCESS CRITERIA — STUDENTS CAN

- ✓ spot repeated subproblems in recursion
- ✓ define state as “what changes”
- ✓ pick the correct cache key
- ✓ convert recursion → memoized recursion
- ✓ write top-down DP for Fibonacci
- ✓ write top-down DP for Min Cost Stairs
- ✓ explain backtracking vs memoization

The Bridge Forward



If you can name the state, you can memoize anything.

THE HABIT TO CARRY OUT OF TODAY

- Hear a problem → ask “what changes?”
- Name the state, then key a cache by it
- Solve once, store, reuse — never recompute

Today you climbed from recursion to memoization.

That same state you named is the index of tomorrow’s table.

NEXT CLASS • DAY 3 PREVIEW

```
TOP-DOWN (today)      BOTTOM-UP (Day 3)

solve(n):              dp[0], dp[1] = base
    if cache[n]: ..    for i in 2..n:
        cache[n] = ..    dp[i] = dp[i-1]
                        + dp[i-2]
    return cache[n]

same state -> same table index
```

Day 3 turns the memo cache into a table you fill in order — no recursion, same idea.